

A Comparison of Stochastic Gradient Descent, Scaled Conjugate Gradient and LeapFrog for the Training of Feedforward Neural Networks

S.L. Johnson

27356485

Department of Computer Science

Stellenbosch University

27356485@sun.ac.za

Code <https://github.com/FearlessSnail/ML-assignment3.git>

Abstract—This study compared three algorithms for the training of feedforward neural networks with one hidden layer: stochastic gradient descent, scaled conjugate gradient and the LeapFrog dynamic method. Five classification and four function approximation problems of varying complexity served as benchmarks. After the optimal number of hidden units and the best control parameters had been determined empirically for each problem, the algorithms were compared over 30 paired runs per problem under an equal budget of computational work. No algorithm generalised significantly better across the nine problems. Stochastic gradient descent attained a validation cost common to all three algorithms with significantly less computational work, whereas the two batch methods required significantly less wall-clock time. Scaled conjugate gradient was insensitive to the control parameters over the grid tested and was concluded to be the most practical of the three algorithms.

I. INTRODUCTION

A feedforward neural network learns a mapping from inputs to outputs through the adjustment of the weights of the network. The adjustment is an unconstrained optimisation problem, and the choice of optimisation algorithm affects both the quality of the trained network and the cost of training. The present study compares three algorithms that rest on different principles. Stochastic gradient descent (SGD) follows the negative gradient of the error on small random subsets of the data, whereas the scaled conjugate gradient (SCG) algorithm of Möller [12] uses second-order information to choose the search direction and the step size. The LeapFrog algorithm of Snyman [18], [19] models the weights as a particle that moves under the force of the negative gradient. SCG and LeapFrog are batch methods, because every step uses the gradient over the full training set.

A fair comparison requires care: each algorithm has to receive the same architecture, the best control parameters for the problem and an equal amount of computational work. The control parameters of an algorithm are the constants that the user sets before training, such as the learning rate. Because every training run also depends on a random data split and on random initial weights, the results have to be tested statistically. The purpose of the study was to identify the best

of the three algorithms under such a controlled comparison. After the optimal number of hidden units and the best control parameters had been determined for nine benchmark problems, the algorithms were compared on 30 fresh paired runs per problem with rank-based statistical tests.

No algorithm generalised significantly better across the nine problems. SGD learned significantly faster per unit of computational work, whereas the two batch methods were significantly faster in wall-clock time. SCG was also insensitive to the control parameters over the grid tested.

The remainder of the report is structured as follows. Section II presents the background on neural networks, the three algorithms and the statistical tests, and Section III describes the implementation. Section IV details the empirical process, Section V presents and discusses the results, and Section VI concludes the study.

II. BACKGROUND

This section provides the theoretical foundation for the study. Feedforward neural networks and generalisation are described first, followed by the three training algorithms and a common measure of computational work. The section closes with the statistical tests used to compare algorithms.

A. Feedforward Neural Networks

A feedforward neural network with one hidden layer maps an input vector $\mathbf{x}$ of d components to an output vector $\mathbf{y}$ of c components through h hidden units [2]. Each unit passes a weighted sum of the values of the preceding layer, referred to as the net input, through an activation function. A bias is the weight of an additional input fixed at one. Let $\mathbf{U}$ denote the $(d+1) \times h$ matrix of weights from the inputs to the hidden units, and $\mathbf{V}$ the $(h+1) \times c$ matrix of weights from the hidden units to the outputs. The network computes

$$\mathbf{z} = f(\mathbf{U}^\top [\mathbf{x}; 1]), \quad \mathbf{y} = g(\mathbf{V}^\top [\mathbf{z}; 1]), \quad (1)$$

where $\mathbf{z}$ denotes the vector of hidden unit outputs and $[\mathbf{x}; 1]$ denotes $\mathbf{x}$ extended with a constant one. The functions f and

g denote the hidden and output activation functions, applied to each component.

The classical activation function is the logistic sigmoid $f(\nu) = 1/(1 + \exp(-\nu))$, where ν denotes the net input of a unit. The sigmoid is smooth and bounded on the interval $(0, 1)$, whereas the rectified linear unit (ReLU), given by $f(\nu) = \max(0, \nu)$, is a piecewise linear alternative. A linear output function suits function approximation with unbounded targets, whereas a sigmoid output function suits classification by keeping each output in the interval $(0, 1)$. Hornik *et al.* [9] showed that one hidden layer with sufficient sigmoid units approximates any continuous function on a bounded domain to any desired accuracy. The number of hidden units therefore determines the capacity of the network: too few units are unable to represent the function, and too many fit the noise in the training data [2].

All $q = (d + 1)h + (h + 1)c$ weights form a single weight vector $\mathbf{w}$. Training minimises the squared error, summed over the outputs and averaged over the n training patterns, given by

$$e(\mathbf{w}) = \frac{1}{2n} \sum_{i=1}^n \|\mathbf{y}_i - \mathbf{t}_i\|^2, \quad (2)$$

where $\mathbf{y}_i$ and $\mathbf{t}_i$ denote the output and the target for pattern i . The cost as a function of the weights is referred to as the error surface. A forward pass computes the outputs of all units for every training pattern, and backpropagation adds a backward pass that propagates the output errors back through the weights to give the gradient $\nabla e(\mathbf{w})$ [16].

B. Generalisation and Overfitting

The purpose of training is a low error on unseen patterns, referred to as the generalisation error [2]. The training error decreases as training continues and as units are added, whereas the generalisation error eventually increases again because the network starts to fit noise. The data are therefore divided into three disjoint sets. The cost is minimised on the training set and the validation set guides every choice. The test set measures the final network once, so that the choices do not bias the estimate of the generalisation error. The training cost and the validation cost are the cost function e evaluated on the training set and on the validation set. Early stopping limits overfitting by retaining the weights with the lowest validation cost seen during training [15].

The validation error is the generalisation error measured on the validation set. A candidate configuration, such as a number of hidden units, is judged on the mean validation error over repeated runs. The standard error of the mean is the sample standard deviation divided by the square root of the number of runs. Because the candidate with the lowest mean tends to be a lucky and needlessly complex winner, the one-standard-error rule selects the simplest candidate with a mean within one standard error of the lowest mean [5]. The standard error in the rule is that of the candidate with the lowest mean.

C. Stochastic Gradient Descent

SGD moves the weights in the direction of the negative gradient, with the full gradient replaced by the gradient over a small random batch of patterns. A pass over the training set is referred to as an epoch, and batches of b patterns give n/b updates per epoch. A momentum term accumulates the previous updates [14], [16]. Let B denote the current batch, e_B the cost over B and k the iteration index. The update is given by

$$\Delta \mathbf{w}_k = \gamma \Delta \mathbf{w}_{k-1} - \eta \nabla e_B(\mathbf{w}_k), \quad \mathbf{w}_{k+1} = \mathbf{w}_k + \Delta \mathbf{w}_k, \quad (3)$$

where $\Delta \mathbf{w}_k$ denotes the weight change, η the learning rate and γ the momentum. A small learning rate slows progress, and a large learning rate causes divergence. The batch size trades the number of updates against how closely each batch gradient matches the full gradient. Because SGD does not adapt the step size, the performance depends directly on η and γ .

D. Scaled Conjugate Gradient

The Hessian matrix $\mathbf{H}$ contains the second derivatives of e with respect to the weights. Conjugate gradient methods minimise a quadratic function of q variables in at most q steps [6]. Step k searches along a direction $\mathbf{p}_k$ conjugate to every earlier direction $\mathbf{p}_j$, that is $\mathbf{p}_k^\top \mathbf{H} \mathbf{p}_j = 0$ for $j < k$, so that a minimisation along a conjugate direction does not undo the minimisation along earlier directions. The error surface of a neural network is not quadratic, however. The Hessian is positive definite when the error curves upward along every direction, a property often lost far from a minimum. A step size computed from the curvature is then unreliable, so classical conjugate gradient methods determine the step size through a line search, a one-dimensional minimisation of the error along the search direction. Møller [12] noted that the line search costs additional evaluations of the error or the gradient in every iteration.

Møller [12] proposed SCG to remove the line search. First, the product of the Hessian with the search direction is approximated through a finite difference of two gradients, given by

$$\mathbf{s}_k = \frac{\nabla e(\mathbf{w}_k + \sigma_k \mathbf{p}_k) - \nabla e(\mathbf{w}_k)}{\sigma_k}, \quad (4)$$

where $\sigma_k = \sigma / \|\mathbf{p}_k\|$ for a small constant σ . The curvature along the search direction is $\mathbf{p}_k^\top \mathbf{H} \mathbf{p}_k$, approximated by $\mathbf{p}_k^\top \mathbf{s}_k$. Second, a scale parameter λ_k is added to the curvature. As in the Levenberg–Marquardt method, the addition acts as if a multiple of the identity matrix were added to the Hessian, and a larger λ_k gives a shorter step. The scaled curvature is

$$\kappa_k = \mathbf{p}_k^\top \mathbf{s}_k + \lambda_k \|\mathbf{p}_k\|^2. \quad (5)$$

The scale parameter is raised whenever $\kappa_k \leq 0$, so that the quadratic model always has a minimum along the search direction. Third, a comparison parameter ρ_k measures the agreement between the actual reduction of the error and the reduction predicted by a quadratic model, the second-order

Taylor approximation of the error along the search direction. A large ρ_k indicates a reliable model and lowers λ_k to allow longer steps, whereas a small ρ_k raises λ_k and shortens the steps.

Algorithm 1 summarises SCG, with the negative gradient $\mathbf{r}_k = -\nabla e(\mathbf{w}_k)$, the step size $\alpha_k = \mu_k / \kappa_k$ and $\mu_k = \mathbf{p}_k^\top \mathbf{r}_k$. A temporary scale parameter $\bar{\lambda}_k$ records the scaling already added to the curvature, so that a rejected step is retried without computing $\mathbf{s}_k$ again. Each iteration costs two gradient evaluations and one error evaluation. Møller [12] recommended $\sigma \leq 10^{-4}$ and an initial scale parameter $\lambda_1 \leq 10^{-6}$. Having found the performance on the 5-bit parity problem not significantly affected by $\sigma \leq 10^{-4}$, Møller concluded that SCG has no user-dependent parameters crucial to success.

E. LeapFrog

Snyman [18] treated minimisation as a problem of mechanics. The weights are the position of a particle of unit mass, the error $e(\mathbf{w})$ is the potential energy of the particle, and the force on the particle is the negative gradient. The particle therefore follows $\ddot{\mathbf{w}} = -\nabla e(\mathbf{w})$, where the double dot denotes the second derivative with respect to time. The kinetic energy of the particle is $\frac{1}{2} \|\dot{\mathbf{w}}\|^2$, where the single dot denotes the first derivative. Because the force field is conservative, the sum of the potential and the kinetic energy is constant, and a decrease in the error is matched by an increase in the kinetic energy. The trajectory is integrated with the leap-frog scheme, given by

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \tau \mathbf{v}_k, \quad \mathbf{v}_{k+1} = \mathbf{v}_k + \tau \mathbf{a}_{k+1}, \quad (6)$$

where $\mathbf{v}_k$ denotes the velocity, $\mathbf{a}_k = -\nabla e(\mathbf{w}_k)$ the acceleration and τ the time step.

An undisturbed particle would oscillate indefinitely, so LeapFrog interferes whenever the particle loses speed, a sign that the particle climbs. The particle then restarts from the midpoint of the last step with a reduced velocity, or from rest after repeated restarts. The original algorithm, LFOP1, limits the step length $\tau \|\mathbf{v}_k\|$ to a maximum δ [18], and the improved version LFOP1(b) adds an automatic control of the time step [19]. The force reverses when the new acceleration points against the previous one, that is when $\mathbf{a}_{k+1}^\top \mathbf{a}_k \leq 0$. After every step shorter than δ and without a reversal, the time step is multiplied by a growth factor that increases by δ_1 after each such step and returns to one after a reversal. The time step halves after m consecutive reversals. Each step of LFOP1(b), summarised in Algorithm 2, costs one gradient evaluation. Snyman used $\tau_0 = 0.5$ and $\delta = 1$ for LFOP1 [18], and added $m = 3$ and $\delta_1 = 0.001$ for LFOP1(b) [19].

F. Computational Work

Iterations of the three algorithms are not comparable, so Møller [12] measured work in complexity units (CUs). One CU is one forward pass over the full training set, and a gradient over the full training set costs three CUs. One SGD epoch therefore costs three CUs regardless of the batch size, as does one LeapFrog step, while one SCG iteration costs about seven.

Algorithm 1 Scaled conjugate gradient

- 1: Set $\mathbf{p}_1 = \mathbf{r}_1$, $\bar{\lambda}_1 = 0$, $k = 1$ and mark the last step as successful
 - 2: **repeat**
 - 3: **if** the last step was successful **then**
 - 4: Compute $\mathbf{s}_k$ and the curvature $\mathbf{p}_k^\top \mathbf{s}_k$
 - 5: **end if**
 - 6: Add $(\lambda_k - \bar{\lambda}_k) \|\mathbf{p}_k\|^2$ to the curvature to obtain κ_k
 - 7: Raise λ_k until the curvature is positive, if $\kappa_k \leq 0$
 - 8: Set $\alpha_k = \mu_k / \kappa_k$ and compute $\rho_k = 2\kappa_k [e(\mathbf{w}_k) - e(\mathbf{w}_k + \alpha_k \mathbf{p}_k)] / \mu_k^2$
 - 9: **if** $\rho_k \geq 0$ **then**
 - 10: Accept $\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha_k \mathbf{p}_k$ and set $\bar{\lambda}_k = 0$
 - 11: Set $\beta_k = (\|\mathbf{r}_{k+1}\|^2 - \mathbf{r}_{k+1}^\top \mathbf{r}_k) / \mu_k$ and $\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$, or $\mathbf{p}_{k+1} = \mathbf{r}_{k+1}$ after every q iterations
 - 12: Divide λ_k by four if $\rho_k \geq 0.75$
 - 13: **else**
 - 14: Reject the step, set $\bar{\lambda}_k = \lambda_k$ and mark the step as unsuccessful
 - 15: **end if**
 - 16: Increase λ_k if $\rho_k < 0.25$, and increase k by one
 - 17: **until** the gradient vanishes or the budget is spent
-

Algorithm 2 LeapFrog LFOP1(b)

- 1: Set $\mathbf{v}_0 = \frac{1}{2} \tau_0 \mathbf{a}_0$ and $\tau = \tau_0$
 - 2: **repeat**
 - 3: **if** the step length $\tau \|\mathbf{v}_k\|$ exceeds δ **then**
 - 4: Scale $\mathbf{v}_k$ so that the step length equals δ
 - 5: **else**
 - 6: Increase the growth factor by δ_1 and multiply τ by the factor
 - 7: **end if**
 - 8: **if** the force reversed direction in m consecutive steps **then**
 - 9: Halve τ , move to the midpoint of the last step and damp the velocity
 - 10: **end if**
 - 11: Set $\mathbf{w}_{k+1} = \mathbf{w}_k + \tau \mathbf{v}_k$ and $\mathbf{v}_{k+1} = \mathbf{v}_k + \tau \mathbf{a}_{k+1}$
 - 12: **if** $\|\mathbf{v}_{k+1}\| \leq \|\mathbf{v}_k\|$ **then**
 - 13: Restart from the midpoint with the velocity $\frac{1}{4}(\mathbf{v}_{k+1} + \mathbf{v}_k)$, or with zero velocity after repeated restarts
 - 14: **end if**
 - 15: Increase k by one
 - 16: **until** the gradient vanishes or the budget is spent
-

G. Statistical Comparison of Algorithms

The data split, the initial weights and the order of the batches are all random, so algorithms are compared over repeated runs. In a paired design, one seed fixes the data split and the initial weights for every algorithm, so that the variation between splits cancels in the differences. The paired t -test assumes normally distributed differences, an assumption that the Shapiro–Wilk test assesses [17]. The Wilcoxon signed-rank test [20] makes no such assumption [1] and compares

the rank sums of the positive and the negative differences. The Hodges–Lehmann estimator, the median of all pairwise averages of the differences, estimates the size and direction of the difference between two algorithms [7]. The procedure of Holm [8] adjusts the probability values of multiple tests to control the probability of any false positive.

Demšar [1] recommended rank-based tests for the comparison of algorithms over multiple problems. The algorithms are ranked on each problem, and the Friedman test [4] assesses whether the mean ranks differ. The Nemenyi test [1], [13] declares two algorithms different when the mean ranks differ by more than a critical difference. For three algorithms and nine problems, the critical difference at a significance level of 0.05 is 1.10 mean ranks [1].

In summary, SGD relies on the gradient alone, SCG adds curvature information without a line search, and LeapFrog follows a damped physical trajectory. CUs make the work of the three algorithms comparable, and rank-based tests compare the algorithms without an assumption of normality.

III. IMPLEMENTATION

This section describes the implementation, starting with the software and followed by the network and the training algorithms.

A. Software

The implementation is written in Python¹, and the NumPy library² provides all numerical computation, including the network and the three training algorithms. The training algorithms are implemented from the source papers rather than taken from a library, so that every algorithm counts work in exactly the same way and is verifiable against the source paper. The scikit-learn library³ supplies five benchmark problems and the stratified division of the data but plays no part in training. The SciPy library⁴ provides the statistical tests, and the Matplotlib library⁵ produces the figures.

B. Network and Training Algorithms

The network stores all weights in one vector, so the training algorithms treat the network as a function from the weights to the cost and the gradient. A comparison with central finite differences confirmed the gradient to a relative error of 4.4×10^{-8} . The initial weights are drawn uniformly from the interval $[-1/\sqrt{\phi}, 1/\sqrt{\phi}]$, where ϕ denotes the number of inputs to a unit, bias included [2]. The interval keeps the initial net inputs in the responsive range of the sigmoid [11]. Every evaluation of the cost or the gradient adds the CUs of the evaluation to one counter, so that the work of all three algorithms is counted in the same way.

The three algorithms share one interface: each receives the initial weights, the cost function and a monitor. SGD reshuffles

the patterns every epoch, while the implementations of SCG and LeapFrog follow the steps of the source papers one by one. SCG additionally stops when the search direction vanishes or the scale parameter exceeds 10^{20} , two cases that the paper does not discuss. The monitor evaluates the training and the validation cost at the end of an iteration, at most once every three CUs. The monitor records the weights with the lowest validation cost and ends the run when the budget is spent. One seed fixes the data split, the initial weights and the order of the SGD batches, so that runs with the same seed are paired and every result is reproducible.

IV. EMPIRICAL PROCESS

This section describes the empirical process. The benchmark problems and the preprocessing of the data come first, followed by the selection of the number of hidden units and of the control parameters. The section closes with the performance criteria and the comparison protocol.

A. Benchmark Problems

Table I lists the nine benchmark problems, chosen to vary in size, dimension, number of classes, non-linearity and noise.

Five problems are classification problems. Iris contains four measurements of 150 flowers of three species. Wine contains 13 chemical measurements of 178 wines from three cultivars⁶, on scales that differ by four orders of magnitude. Cancer contains 30 features of cell nuclei from 569 breast tissue samples, labelled malignant or benign⁷. Digits contains 1 797 images of handwritten digits of eight by eight pixels, in 10 classes, in the version distributed with scikit-learn, the test portion of the original data set⁸. Spirals is a version of the two-spirals problem [10], with 400 points on two interlocking spirals of 1.25 turns separated by a highly non-linear boundary.

Four problems are function approximation problems. Let x_1 and x_2 denote the first two components of the input vector. Sinc samples $\sin(x_1)/x_1$ at 300 points on the interval $[-10, 10]$. Sin2D samples $\sin(2\pi x_1) \cos(2\pi x_2)$ at 600 points on the unit square. Both add Gaussian noise with a standard deviation of 0.05. Friedman1 samples the 10-input test function of Friedman [3] at 600 points, with five irrelevant inputs and Gaussian noise of standard deviation one. Diabetes contains 10 measurements of 442 patients and the progression of the disease after one year.

B. Data Preprocessing and Network Configuration

Every run divided the data into 60 percent training, 20 percent validation and 20 percent test patterns, stratified for classification so that each set kept the class proportions. Every input was standardised to zero mean and unit variance with the statistics of the training set alone. Inputs constant on the training set, such as pixels of Digits that are blank in every training image, were centred but not scaled. Standardisation prevents large inputs from saturating the sigmoid units [2]

¹<https://www.python.org>

²<https://numpy.org>

³<https://scikit-learn.org>

⁴<https://scipy.org>

⁵<https://matplotlib.org>

⁶<https://archive.ics.uci.edu/dataset/109/wine>

⁷<https://archive.ics.uci.edu/dataset/17>

⁸<https://archive.ics.uci.edu/dataset/80>

TABLE I
BENCHMARK PROBLEMS

Problem	Task	n	d	c	Budget
Iris	classification	150	4	3	600
Wine	classification	178	13	3	900
Cancer	classification	569	30	2	900
Digits	classification	1797	64	10	2400
Spirals	classification	400	2	2	6000
Sinc	regression	300	1	1	3000
Sin2D	regression	600	2	1	4000
Friedman1	regression	600	10	1	2000
Diabetes	regression	442	10	1	900

and improves the conditioning of the error surface [11]. The conditioning is the ratio of the largest to the smallest curvature of the error surface, given by the eigenvalues of the Hessian [11]. A poorly conditioned surface forms long narrow valleys, where no single step size suits every direction. The effect of a weight change on the net input depends on the scale of the inputs. Standardisation therefore also gives the learning rate of SGD and the step limit of LeapFrog a comparable effect across problems.

Classification targets were coded with one output per class, at 0.9 for the true class and 0.1 for the others. Because a sigmoid attains zero or one only in the limit, targets of zero and one would drive the weights towards infinity [2], [11]. No problem contained missing values, and the only nominal input was the binary sex variable of Diabetes.

Every network had one hidden layer of sigmoid units, because SCG and LeapFrog both assume a smooth cost function. The output units were sigmoid for classification and linear for function approximation, and every problem used the averaged sum of squared errors as the cost function. Every run on a problem received the same budget of CUs, listed in Table I. On seven problems, the budgets left ample room beyond the typical point of convergence: over the final tenth of the budget, the median improvement of the lowest validation cost over the comparison runs was zero. On Sinc and Sin2D, the median improvement over the final tenth was up to 3 percent, so the results on the two problems reflect the budget as well as the algorithms.

C. Selection of the Number of Hidden Units

For each size, the validation error was averaged over repeated runs. The size with the lowest mean set a threshold one standard error above that mean, and the optimal number of hidden units h^* was the smallest size with a mean at or below the threshold. For classification, the validation error is the fraction of misclassified validation patterns. For function approximation, the validation error is the mean squared error (MSE) on the validation set. Early stopping and the tie-break of Section IV-D use the validation cost instead. The validation error measures generalisation, and the mean over repeated runs estimates the expected error. The smallest size at or below the threshold avoids a choice based on differences below the level of noise.

Stage A trained networks of 1, 2, 4, 6, 8, 12, 16, 24 and 32 hidden units, a grid dense at the small sizes where the error changes fastest. Each algorithm trained every size with five seeds, at the default control parameters of Table II, because the control parameters were unknown before the architecture was chosen. The validation errors of the three algorithms were pooled, so that one size served all three algorithms and the comparison held the architecture fixed. Stage A2 repeated stage A with the control parameters fixed at the values that stage B tuned at h^* , to test whether the choice depended on the order of the two stages.

D. Selection of the Control Parameters

The best control parameters of an algorithm on a problem were chosen with the same rule. At h^* , the setting with the lowest mean validation error set a threshold one standard error above that mean. Among the settings at or below the threshold, the setting with the least mean work to the lowest validation cost of a run was preferred. Stage B evaluated every combination in Table II with 10 seeds on every problem. A full grid was necessary because the parameters interact: the effective step of SGD, for example, grows approximately as $\eta/(1 - \gamma)$. Each range extended from conservative to aggressive values, on an approximately logarithmic scale for η , b , σ , λ_1 , τ_0 , δ and δ_1 . A first pass with five seeds selected a setting of SGD close to divergence on Sin2D, so the number of seeds was raised to 10 for every problem and algorithm.

E. Performance Criteria and Comparison Protocol

Four performance criteria were defined before the comparison. The primary criterion was the generalisation error on the test set. For classification, the generalisation error is the misclassification rate, equal to one minus the accuracy, where the accuracy is the fraction of test patterns classified correctly. The MSE alone is a poor measure of accuracy for classification [2]. For function approximation, the generalisation error is the MSE on the test set. The second criterion was the MSE on the test set for classification as well, a finer measure than the misclassification rate. The third criterion was the work to target, the number of CUs a run required to attain the target of the split. On each split, every algorithm attained a lowest validation cost, and the target of the split was the highest of the three. A target per split adapts the criterion to the difficulty of the split, and every run attains the target by construction. The fourth criterion was the wall-clock time for the full budget.

Stage C compared the algorithms on 30 seeds that no earlier stage had used, with each algorithm at h^* , the tuned control parameters and the budget of the problem. Stages A, B, A2 and C comprised 1215, 11610, 1215 and 810 runs. Within each problem, the Wilcoxon signed-rank test compared every pair of algorithms, with the Holm correction over the three pairs and a significance level of 0.05. The rank-based test was preferred to the paired t -test because the rank-based test does not assume normality [1]. The Shapiro–Wilk test supported the choice, rejecting normality of the paired differences in 16 of the 27 comparisons of generalisation error. Differences in

TABLE II
GRIDS OF CONTROL PARAMETERS

Algorithm	Parameter	Values	Default
SGD	η	0.05, 0.1, 0.5, 1, 2	0.1
	γ	0, 0.5, 0.9	0.9
	b	8, 32, 128	32
SCG	σ	$10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}$	10^{-4}
	λ_1	$10^{-8}, 10^{-6}, 10^{-4}$	10^{-6}
LeapFrog	τ_0	0.1, 0.5, 1	0.5
	δ	0.5, 2, 10, 50	1
	m	3, 5, 8	3
	δ_1	0.001, 0.01	0.001

misclassification rate are discrete and often tied, a property that alone departs from normality. Across the nine problems, the Friedman test and the Nemenyi critical difference compared the mean ranks, with each algorithm ranked on the mean of each criterion over the 30 runs. The complete process was also repeated with ReLUs in the hidden layer, with the same grids of control parameters and a grid of sizes extended by 48, 64, 96 and 128 units. The repetition tested whether the conclusions survived a cost function that is not smooth.

In summary, the number of hidden units and the control parameters were chosen on validation data with the one-standard-error rule and full grids. The comparison used 30 fresh paired runs per problem, four criteria and rank-based tests.

V. RESULTS AND DISCUSSION

This section presents and discusses the results, beginning with the optimal numbers of hidden units and the tuned control parameters. The generalisation error, the speed of learning and the overall comparison follow.

A. Optimal Number of Hidden Units

Figure 1 shows the mean validation error against the number of hidden units, and Table III lists the chosen sizes. The error fell steeply while the network was too small and then flattened. On Iris, Spirals, Sinc and Friedman1, the error rose again for the largest networks.

The chosen sizes reflected the complexity of the problems. The mean validation error of Cancer varied by less than one misclassified validation pattern across all sizes, so one unit sufficed for the nearly linearly separable problem. Iris required two units, with an error of 0.136 for one unit against 0.018 for two. Every output of a network with one hidden unit is a monotone function of the same net input. The middle species of Iris, however, requires a high output between two low outputs, a pattern that no monotone function produces. Spirals required 16 units and Digits 24. The individual algorithms preferred different sizes on the flat curves: on Diabetes, the lowest mean occurred at 16 units for SGD, eight for SCG and 24 for LeapFrog. Pooling the algorithms reduced the influence of noise on the choice.

Stage A2 confirmed the choice on seven of the nine problems, and both changes occurred on flat curves. On Sinc,

TABLE III
OPTIMAL NUMBER OF HIDDEN UNITS

Problem	Stage A					A2
	h^*	Pooled	SGD	SCG	LeapFrog	h^*
Iris	2	4	4	2	2	2
Wine	6	6	8	6	6	6
Cancer	1	1	1	16	12	16
Digits	24	32	32	32	24	24
Spirals	16	16	16	32	32	16
Sinc	6	12	12	8	12	8
Sin2D	12	12	16	12	12	12
Friedman1	8	8	6	8	8	8
Diabetes	8	32	16	8	24	8

the choice moved from six to eight units because six units missed the threshold by 0.001. On Cancer, the choice moved from one to 16 units, with every size performing equally well. The comparison kept the sizes of stage A. Stage A2 also exposed a weakness of SGD: on Sin2D, Friedman1 and Diabetes, the learning rate tuned at h^* failed on larger networks (Figure 2(b)).

Figure 2 explains the failure. Let ω denote the largest eigenvalue of the Hessian with respect to the output weights. On a quadratic function, gradient descent with momentum converges only if $\eta\omega < 2(1 + \gamma)$ [14], so the stability ratio $\eta\omega/[2(1 + \gamma)]$ must stay below one. For a linear output, the Hessian with respect to the output weights is the mean outer product of the hidden outputs, so ω grows as hidden units are added. Figure 2(a) shows the stability ratio at the initial weights for the tuned parameters of SGD, and Figure 2(b) shows the validation error of tuned SGD in stage A2, with SCG dotted. On Sin2D and Friedman1, the ratio exceeded one only at 32 units, where the error of SGD rose to the level of an untrained network. On Diabetes, the ratio already attained one at eight units, and the error of SGD rose from 12 units onwards. SCG and LeapFrog adapt the step to the error surface and therefore did not suffer from the change of size.

B. Control Parameters

Table IV lists the chosen control parameters, and the column labelled tied counts the settings at or below the threshold of Section IV-D. A high count indicates insensitivity to the parameters. Table V measures sensitivity in two ways, each as the median and the maximum over the nine problems. The first fixes one parameter at a value and tunes the others. The parameter-fixed columns give how far the best mean validation error attainable at the worst such value lies above the overall best. The second is the spread between the best and the worst setting of the whole grid. A large spread with small values for single parameters indicates an interaction between parameters. For classification, the values are differences in misclassification rate. For function approximation, the values are differences in MSE on standardised targets, so a spread above one means that the worst setting did worse than predicting the mean.

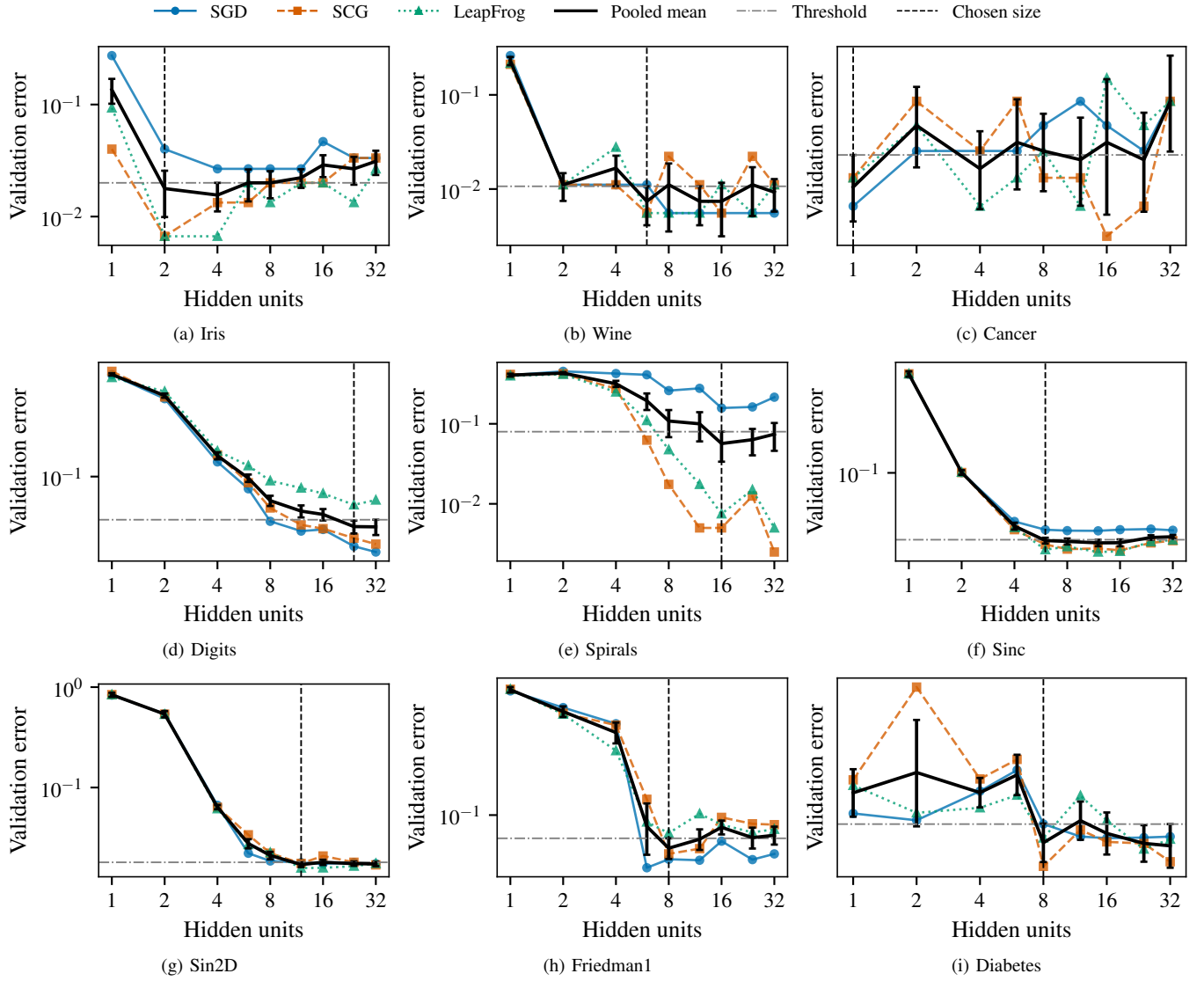


Fig. 1. Mean validation error against the number of hidden units

TABLE IV
CHOSEN CONTROL PARAMETERS

Problem	SGD				SCG			LeapFrog				
	η	γ	b	Tied	σ	λ_1	Tied	π_0	δ	m	δ_1	Tied
Iris	2	0.9	8	4/45	10^{-3}	10^{-4}	12/12	1	50	8	0.01	14/72
Wine	1	0.9	8	33/45	10^{-3}	10^{-6}	12/12	0.1	10	8	0.01	14/72
Cancer	2	0.5	8	14/45	10^{-3}	10^{-6}	12/12	1	2	8	0.01	6/72
Digits	2	0.5	8	28/45	10^{-5}	10^{-6}	12/12	0.5	10	5	0.01	13/72
Spirals	1	0.9	8	1/45	10^{-5}	10^{-6}	4/12	1	10	3	0.01	3/72
Sinc	0.1	0.9	8	3/45	10^{-6}	10^{-8}	11/12	0.1	2	3	10^{-3}	17/72
Sin2D	0.5	0.9	128	2/45	10^{-3}	10^{-6}	6/12	0.1	2	8	10^{-3}	3/72
Friedman1	0.5	0.9	128	2/45	10^{-5}	10^{-8}	9/12	1	0.5	8	0.01	24/72
Diabetes	1	0.5	32	20/45	10^{-4}	10^{-4}	12/12	1	0.5	3	10^{-3}	10/72

SGD preferred aggressive settings, with large learning rates and small batches, and was sensitive to the settings. A learning rate of at least 0.5 was chosen on eight problems, and a batch size of eight, the size with the most updates per CU, on six.

On Spirals, only one of the 45 settings lay at or below the threshold. The learning rate was the decisive parameter, with the largest parameter-fixed value in Table V, 0.906 on Sin2D. At a learning rate of two, 22.5 percent of the runs never

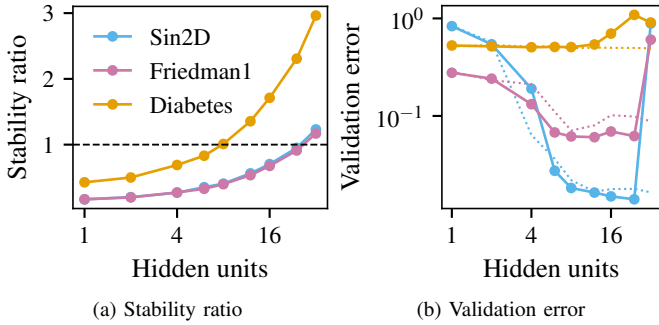


Fig. 2. Stability of tuned stochastic gradient descent

TABLE V
SENSITIVITY TO THE CONTROL PARAMETERS

Algorithm	Parameter	Parameter fixed		Whole grid	
		Median	Maximum	Median	Maximum
SGD	η	0.0575	0.9062	0.5800	1.1526
	γ	0.0042	0.0150		
	b	0.0044	0.0233		
SCG	σ	0.0009	0.0033	0.0022	0.0177
	λ_1	0.0007	0.0025		
LeapFrog	τ_0	0.0033	0.0067	0.0267	0.7463
	δ	0.0053	0.0164		
	m	0.0025	0.0147		
	δ_1	0.0033	0.0081		

improved on the initial weights, so the best settings of SGD lay close to the boundary of divergence.

SCG was effectively free of parameters: over the whole grid, the mean validation error varied by at most 0.018 on any problem, and all 12 settings were tied on five problems. The present grid confirmed the finding of Møller [12] that the performance was insensitive to σ and extended the finding to $\sigma = 10^{-3}$ and to λ_1 . The results of Møller already hinted at the first extension, with $\sigma = 10^{-3}$ giving the lowest mean number of CUs on parity. The scale parameter adapts during training, so the initial values affect only the first iterations.

LeapFrog was insensitive to each parameter alone, with parameter-fixed values of at most 0.016, but the spread over the whole grid attained 0.75 on Sinc and 0.70 on Sin2D. The poor settings combined the largest step limit of 50 with $m = 8$, the most reversals allowed before the time step halves. On Sinc, the combination gave a mean validation error of 0.528, against 0.028 for a step limit of 0.5. A long permitted step and slow damping let the particle swing across the oscillations of the target function. Snyman [19] suggested that a large step limit was advantageous on the test functions, but on neural networks a large step limit was harmful when combined with slow damping.

C. Generalisation Error

Table VI reports the generalisation error, and Table VII the pairwise tests. Each pairwise cell gives the significantly better algorithm or a tie, and the number of runs won by each algorithm of the pair. The standard deviations over the 30 runs

were of the same size as the differences between the means, and 20 of the 27 pairwise comparisons were ties.

The seven significant differences had identifiable causes. SGD and SCG beat LeapFrog on Digits, with errors of 0.033 and 0.034 against 0.053. The median training cost of LeapFrog was 0.057 against 0.009 for SGD, so the cause was a failure of optimisation rather than overfitting. Figure 3 shows one run of LeapFrog on Digits and on Sin2D at the tuned parameters. The training cost on Digits stopped improving within the first tenth of the budget, while the time step grew to 100, against at most 1.4 on Sin2D. With the step limit active on 45 percent of the steps, LeapFrog degenerated into a method of fixed step length.

Both batch methods beat SGD on Sinc, with a MSE of about 0.025 against 0.029. SGD used batches of eight patterns on Sinc, and ended at a median training cost of 0.0133 against 0.0111 for SCG. The gradient of a small batch keeps the weights moving around the minimum, so SGD settled less precisely. LeapFrog beat both other algorithms on Sin2D, with the lowest training cost of the three. SCG beat SGD on Spirals, at 0.012 against 0.020.

D. Speed of Learning and Wall-Clock Time

Table VIII reports the median work to target and the mean wall-clock time. SGD attained the target first on eight of the nine problems and won 13 of the 18 pairwise comparisons of work that involved SGD and tied the other five. Figure 4 shows the learning curves of four problems. On Digits, SGD attained the target within two epochs, while LeapFrog levelled off above the other two algorithms. One epoch of SGD with batches of eight patterns performs $n/8$ updates for three CUs, against one update for a LeapFrog step. Early in training, the gradient of a small batch points in nearly the same direction as the full gradient, so each update makes almost full progress. The batch methods started slowly and caught up later, with SCG ending lowest on Spirals and LeapFrog on Sin2D.

The advantage of SGD narrowed on Sinc and Sin2D, where every algorithm required more than 1800 CUs to attain the target. The target therefore lay close to the lowest validation cost attainable within the budget. Møller [12] noted that the quadratic approximation improves near a minimum and gives a conjugate gradient method fast convergence. On Sinc, SCG attained the target with less work than SGD.

The order reversed in wall-clock time, with SGD the slowest algorithm on every problem by a factor between 1.4 on Sin2D and 24.7 on Cancer. An epoch of SGD performs many small matrix products where a batch method performs one large product, and each product carries a fixed overhead that the CUs ignore.

E. Overall Comparison

Table IX reports the tests across the nine problems and the mean ranks. Bold marks the lowest mean rank of a criterion, and a dagger marks a mean rank that exceeds the lowest by more than the critical difference of 1.10. The work to target and the wall-clock time differed significantly, whereas

TABLE VI
GENERALISATION ERROR OVER 30 RUNS

Problem	SGD	SCG	LeapFrog
Iris	0.0411 $\pm$ 0.0368	0.0433 $\pm$ 0.0363	0.0489 $\pm$ 0.0358
Wine	0.0204 $\pm$ 0.0218	0.0194 $\pm$ 0.0254	0.0269 $\pm$ 0.0225
Cancer	0.0281 $\pm$ 0.0153	0.0275 $\pm$ 0.0178	0.0289 $\pm$ 0.0148
Digits	0.0329 $\pm$ 0.0097	0.0344 $\pm$ 0.0092	0.0531 $\pm$ 0.0177
Spirals	0.0200 $\pm$ 0.0238	0.0117 $\pm$ 0.0164	0.0254 $\pm$ 0.0691
Sinc	0.0286 $\pm$ 0.0063	0.0245 $\pm$ 0.0053	0.0251 $\pm$ 0.0054
Sin2D	0.0194 $\pm$ 0.0038	0.0196 $\pm$ 0.0042	0.0176 $\pm$ 0.0044
Friedman1	0.0770 $\pm$ 0.0129	0.0977 $\pm$ 0.0663	0.0730 $\pm$ 0.0171
Diabetes	0.5447 $\pm$ 0.0989	0.5453 $\pm$ 0.0958	0.5475 $\pm$ 0.0897

TABLE VII
PAIRWISE COMPARISON OF GENERALISATION ERROR

Problem	SCG, SGD	LeapFrog, SGD	LeapFrog, SCG
Iris	tie 3:5	tie 4:11	tie 3:7
Wine	tie 4:2	tie 6:11	tie 6:10
Cancer	tie 8:7	tie 10:11	tie 8:13
Digits	tie 9:17	SGD 2:25	SCG 5:25
Spirals	SCG 14:2	tie 11:11	tie 6:13
Sinc	SCG 25:5	LeapFrog 24:6	tie 13:17
Sin2D	tie 15:15	LeapFrog 23:7	LeapFrog 22:8
Friedman1	tie 17:13	tie 18:12	tie 21:9
Diabetes	tie 17:13	tie 17:13	tie 15:15

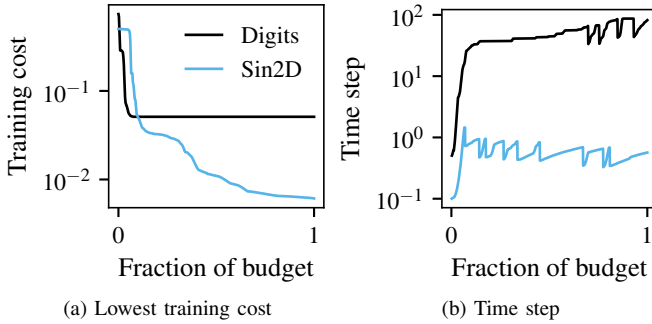


Fig. 3. One LeapFrog run on Digits and Sin2D

the generalisation error and the MSE on the test set did not. On generalisation error, SCG and SGD shared a mean rank of 1.78. Within the problems, SCG won three significant comparisons and lost one, LeapFrog won three and lost two, and SGD won one and lost four. On work to target, SGD was significantly faster than LeapFrog, while the lead of SGD over SCG, at 1.00 mean rank, fell just within the critical difference of 1.10. On wall-clock time, both batch methods were significantly faster than SGD.

The best algorithm therefore depends on the scarce resource. SGD is the best choice when gradient evaluations are scarce and tuning is affordable, whereas the batch methods are the better choice when time is scarce. Between the two batch methods, the effort of tuning decides. The default parameters of SCG were statistically as good as the best setting on every problem, and SCG did not fail on Digits. SCG was therefore the most practical of the three algorithms. The finding

TABLE VIII
MEDIAN WORK TO TARGET AND MEAN WALL-CLOCK TIME

Problem	Complexity units			Seconds		
	SGD	SCG	LeapFrog	SGD	SCG	LeapFrog
Iris	30	186	213	0.16	0.02	0.02
Wine	12	73	138	0.30	0.03	0.03
Cancer	24	84	90	0.94	0.04	0.04
Digits	6	89	111	9.38	2.77	3.09
Spirals	710	1383	1473	4.48	0.42	0.44
Sinc	2133	1836	2535	1.46	0.11	0.11
Sin2D	2499	3640	3068	0.49	0.41	0.35
Friedman1	384	1028	1017	0.28	0.14	0.12
Diabetes	28	80	104	0.19	0.04	0.04

TABLE IX
TESTS ACROSS THE NINE PROBLEMS AND MEAN RANKS

Criterion	Probability value	Mean rank		
		SGD	SCG	LeapFrog
Generalisation	0.2636	1.78	1.78	2.44
Squared error	0.1690	1.56	2.00	2.44
Work to target	0.0018	1.11	2.11	2.78 [†]
Wall-clock time	0.0009	3.00 [†]	1.67	1.33

that SGD needed less work than SCG does not contradict Møller [12], who found SCG to need between 8.4 and 23.4 times fewer CUs than backpropagation. The backpropagation of Møller updated the weights once per pass over the data, at a learning rate set by hand and a momentum of 0.9. The SGD of the present study used small batches and was tuned over 45 settings, a far stronger baseline.

The repetition with ReLUs confirmed that no algorithm generalised significantly better, at a Friedman probability value of 0.46, and the ranks on wall-clock time were identical to the ranks of the sigmoid study. The advantage of SGD in work to target lost significance, with a probability value of 0.24, so the fast start of SGD depended on the activation function. The ReLU networks required more hidden units on seven problems, up to 96 on Sinc and Sin2D, because a piecewise linear network approximates a smooth curve with many linear pieces. The ReLUs halved the error on Digits for all three algorithms, whereas the sigmoid units were better in five of the nine algorithm–problem pairs of the smooth problems Sinc, Sin2D and Friedman1.

In summary, the optimal number of hidden units ranged from one to 24. SCG was free of parameters in practice, whereas LeapFrog suffered from one interaction of parameters and SGD was sensitive to the learning rate. No algorithm generalised significantly better across the nine problems. SGD was the fastest per unit of work, and the batch methods were the fastest in wall-clock time.

VI. CONCLUSION

The study compared stochastic gradient descent (SGD), scaled conjugate gradient (SCG) and LeapFrog for the training of feedforward neural networks with one hidden layer, on five

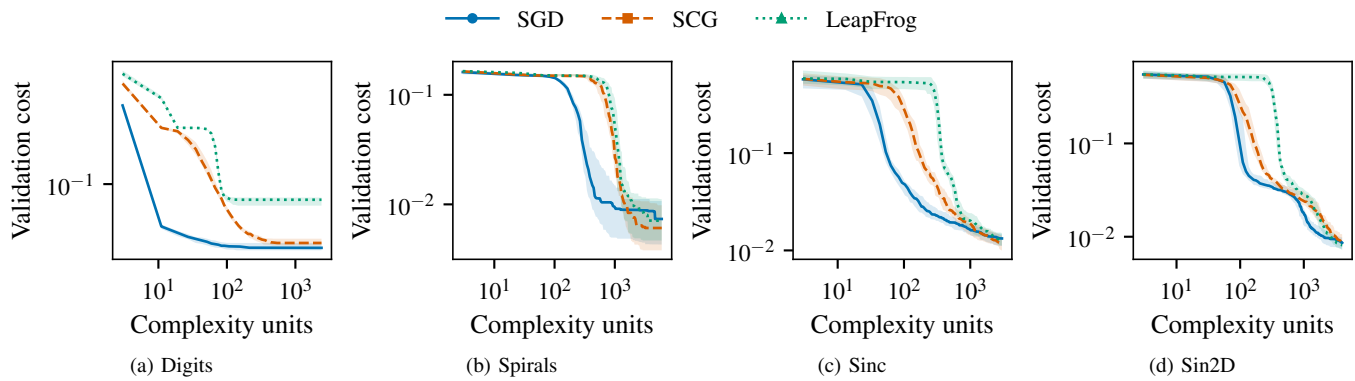


Fig. 4. Median lowest validation cost against work, with the interquartile range shaded

classification and four function approximation problems. After the number of hidden units and the control parameters had been chosen on validation data, the algorithms were compared on 30 fresh paired runs per problem under an equal budget of complexity units (CUs).

No algorithm generalised significantly better across the nine problems. SGD attained a validation cost common to all three algorithms with significantly less work, but required significantly more wall-clock time. SCG was insensitive to the control parameters over the grid tested, and the default parameters of SCG were statistically as good as the best setting on every problem. The tuned learning rate of SGD lay close to the boundary of stability and failed on larger networks. LeapFrog failed to optimise the large Digits network because the step limit dominated the trajectory. SCG was concluded to be the most practical algorithm for the problems studied, and the absence of a significant difference in generalisation held for rectified linear units (ReLUs) as well.

Future work could address four questions. A LeapFrog step limit that scales with the number of weights could remove the failure on large networks. A comparison over a range of budgets would show where the fast start of SGD stops mattering. A learning rate that adapts to the curvature would strengthen the SGD baseline. A cross-entropy cost function would test whether the conclusions depend on the cost function.

REFERENCES

- [1] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.
- [2] A. P. Engelbrecht, *Computational Intelligence: An Introduction*, 2nd ed. Chichester, England: John Wiley & Sons, 2007.
- [3] J. H. Friedman, “Multivariate adaptive regression splines,” *The Annals of Statistics*, vol. 19, no. 1, pp. 1–67, 1991.
- [4] M. Friedman, “The use of ranks to avoid the assumption of normality implicit in the analysis of variance,” *Journal of the American Statistical Association*, vol. 32, no. 200, pp. 675–701, 1937.
- [5] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York: Springer, 2009.
- [6] M. R. Hestenes and E. Stiefel, “Methods of conjugate gradients for solving linear systems,” *Journal of Research of the National Bureau of Standards*, vol. 49, no. 6, pp. 409–436, 1952.
- [7] J. L. Hodges and E. L. Lehmann, “Estimates of location based on rank tests,” *The Annals of Mathematical Statistics*, vol. 34, no. 2, pp. 598–611, 1963.
- [8] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [9] K. Hornik, M. Stinchcombe, and H. White, “Multilayer feedforward networks are universal approximators,” *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989.
- [10] K. J. Lang and M. J. Witbrock, “Learning to tell two spirals apart,” in *Proceedings of the 1988 Connectionist Models Summer School*. San Mateo, California: Morgan Kaufmann, 1988, pp. 52–59.
- [11] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, “Efficient BackProp,” in *Neural Networks: Tricks of the Trade*, ser. Lecture Notes in Computer Science. Berlin: Springer, 1998, vol. 1524, pp. 9–50.
- [12] M. F. Møller, “A scaled conjugate gradient algorithm for fast supervised learning,” *Neural Networks*, vol. 6, no. 4, pp. 525–533, 1993.
- [13] P. B. Nemenyi, “Distribution-free multiple comparisons,” Ph.D. dissertation, Princeton University, Princeton, New Jersey, 1963.
- [14] B. T. Polyak, “Some methods of speeding up the convergence of iteration methods,” *USSR Computational Mathematics and Mathematical Physics*, vol. 4, no. 5, pp. 1–17, 1964.
- [15] L. Prechelt, “Early stopping – but when?” in *Neural Networks: Tricks of the Trade*, ser. Lecture Notes in Computer Science. Berlin: Springer, 1998, vol. 1524, pp. 55–69.
- [16] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [17] S. S. Shapiro and M. B. Wilk, “An analysis of variance test for normality (complete samples),” *Biometrika*, vol. 52, no. 3–4, pp. 591–611, 1965.
- [18] J. A. Snyman, “A new and dynamic method for unconstrained minimization,” *Applied Mathematical Modelling*, vol. 6, no. 6, pp. 449–462, 1982.
- [19] —, “An improved version of the original leap-frog dynamic method for unconstrained minimization: LFOP1(b),” *Applied Mathematical Modelling*, vol. 7, no. 3, pp. 216–218, 1983.
- [20] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.